7 - Java Classes and Objects

Java is strictly an **Object Oriented Programming (OOP)** language. The Object Oriented paradigm follows a few basic principles from which emerge powerful capabilities to extensively structure components and how they are composed together to build a modern application. The fundamental structural element all OOP languages are built on is the class.

Class

At its most basic a class is a container. Programming language developers recognized a need to compose multiple variables together into a single structure to reduce the management and naming burden on early developers. Consider a rectangle which is defined by two independent variables: width and height.

In early languages a rectangle's width and height would need to be managed independently of one another even though they don't have any meaning without the other. Operations on a rectangle would generally need all of the rectangle data in order to perform their roles correctly, *e.g.* a rectangle's area cannot be computed without both the width and height. Functions intended to operate on rectangles would therefore require multiple parameters and more complex ideas and their associated functions might have tens or even hundreds of parameters.

Additionally if the labels `width` and `height` are chosen for a specific rectangle, they might not be available for reuse for other rectangles or even for use in completely different contexts. This might pose a non-trivial naming problem where a program consisting of multiple rectangles might instead use labels like `rect1_width`, `rect1_height`, `rect2_width`, and `rect2_height` to ensure data is easily distinguishable and readable to the programmer. Again for complex ideas and associated functions, labels might become extremely long to make them distinguishable from one another.

```
// C does not have classes. In C, labels tend to be more descriptive and longer,
// function parameter lists tend to be longer and more discrete, and variables need
// much more consideration and management

float rect1_width = 1.0;
float rect1_height = 2.0;
float rect2_width = 5.0;
float rect2_height = 10.0;
float circle1_radius = 1.0;
float circle2_radius = 2.0;

float area_rectangle( float width, float height ) {...}
float perimeter_rectangle( float width, float height ) {...}
float area_circle( float radius ) {...}
float perimeter_circle( float radius ) {...}
```

We might try to resolve this by packing multiple values together into an array which might allow us to pass one parameter containing multiple values; however, few complex ideas have variables of a uniform type, and it is, at the very least, convoluted to put different types into a single array. Additionally, it becomes difficult to keep track of which field, *i.e.* index, represents what in the array for a sufficiently complex idea. However, as difficult as it

might be to pack data into an array, this concept does form the basis of the idea of a "data structure" because a programming language can provide a purpose built type that packs different variables of various types into a single container.

Before the development of OOP languages, the C procedural language introduced a container type via the `struct` type that acts as a data structure; however, the `struct` type is only capable of composing data into the container and has no capacity to compose functions into the same container.

```
// C structs improved the ability to group and contain data
struct rectangle {
    float width;
    float height;
};

struct circle {
    float radius;
};

struct rect1;
struct rect2;
struct circle1;
struct circle2;

rect1.width = 1.0;
rect1.height = 2.0;
rect2.width = 5.0;
rect2.height = 10.0;
circle1.radius = 1.0;
circle2.radius = 2.0;

// But they do not facilitate close association of functions to their data
float area( struct rectangle* r ) {...}
float perimeter( struct rectangle* r ) {...}
float area( struct circle* c ) {...}
float perimeter( struct circle* c ) {...}
```

The `class` type introduced by OOP extends the data structure concept to allow both data and operations associated with the structure to be collected into a single container type. We generally call an operation, *i.e.* a function, associated with a class a **method**, and we generally call a variable associated with a class a **property**, **field**, or **instance variable**.

The term `class` is short for *classification* which means to associate similar elements based on shared characteristics. A class generally acts as both as a container and a blueprint. From the class, we can create an instance of an object that has its own unique copies of the instance variables defined in the class blueprint.

Object Creation

To create a new instance of an object in Java, in the majority of cases, we must use the `new` keyword.

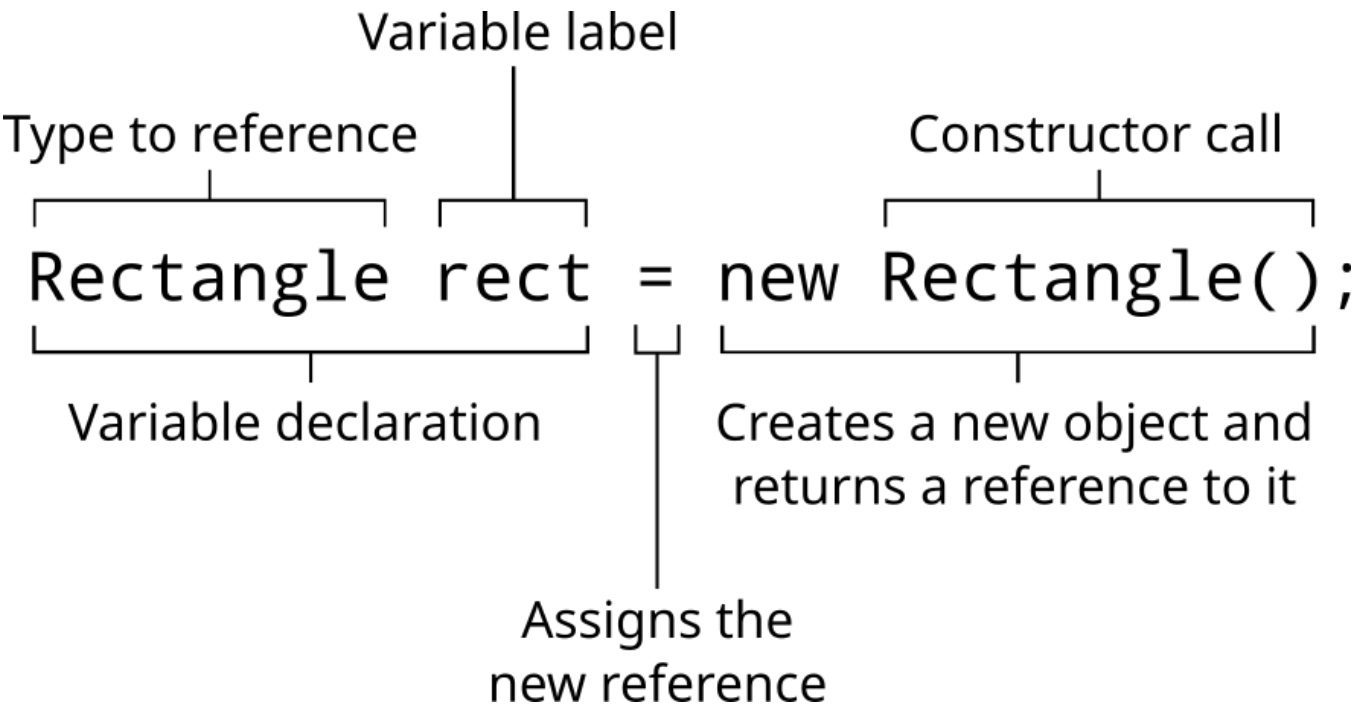
First consider the `Rectangle` class:

```
class Rectangle {
    int width;
    int height;
}
```

The following `RectangleEx1` program creates a `Rectangle` instance which we generally call an object, hence the *object* term in *object oriented programming*:

```
public class RectangleEx1 {
    public static void main (String[] args) {
        Rectangle rect = new Rectangle();
        ...
    }
}
```

The above program combines the declaration with the allocation and construction of the object; however, these are distinct steps. In fact, this simple statement contains considerable complexity as illustrated in the following diagram:



The `RectangleEx2` program illustrates the declaration of the `rectangle` reference type as a separate step from allocation and construction of an associated `Rectangle` instance. The `RectangleEx2` program is generally equivalent to the `RectangleEx1` program:

```
public class RectangleEx2 {
    public static void main (String[] args) {
        Rectangle rect;
        rect = new Rectangle();
        ...
    }
}
```

Recall that there are two distinct elements involved with reference types: the reference variable itself and object instance that it refers to. `RectangleEx2` illustrates these two distinct elements. The `Rectangle rect;` statement represents the declaration of a reference type that is allowed to point only to objects that have been generated from the `Rectangle` class blueprint. We know from our discussion on default initialization of reference types that this statement is actually equivalent to `Rectangle rect = null;`. The subsequent statement `rect = new Rectangle();` triggers the creation of a new `Rectangle` object on the heap, returns a reference to that newly created `Rectangle` object, and assigns the local `rect` variable on the stack a reference to the newly created `Rectangle` object.

Constructors

In Java, when creating an object from a class blueprint, the programmer must provide initial state for the object. This may be done implicitly or explicitly, but initial state of the object is always determined by the constructor called at object creation.

A constructor is NOT a function. While there are some syntactical similarities, a constructor is distinctly different from a method/function. In terms of syntax, a constructor does not have a return type in its declaration, cannot return a value, and shares its name with the class name while a method must have a return type in its declaration, may return a value subject to its declared return type, and can NOT share its name with class name. In terms of invocation, a constructor can only be invoked (called) when preceded by the `new` keyword during object creation while a method can be invoked anywhere the object is valid after the object has been created given a legal reference to the object.

Recall the `Rectangle` class proposed in the previous section:

```
class Rectangle {
    int width;
    int height;
}
```

Note that this version of the class does not explicitly provide a constructor; however, a constructor must be called on object creation. How could a `Rectangle rect = new Rectangle();` statement legal if there is no constructor defined by the `Rectangle` class? Java provides a **default constructor** if the class developer does not explicitly defined a custom constructor. In other words, the above class actually has the following implementation even though the class designer did not provide a constructor:

```
class Rectangle {
    int width;
    int height;

    /* The default constructor would look like the following if it was explicitly defined
    Rectangle() {
        width = 0;
        height = 0;
    }
    But it is commented here to remind us that this code was not written by us */
}
```

The default constructor is a **no parameter constructor** or **parameterless constructor** meaning it takes no inputs hence why the parentheses are empty in the `Rectangle rect = new Rectangle();` default example.

If the class developer implements a constructor, then the default constructor is not provided by Java. The intent of the default constructor is to allow rapid development of a class and it follows established design principles and assumptions associated with Java. Recall that Java already assumes that if an initial value is not provided to a variable, a default value will be assigned. Assigning a default constructor is just this same principle extended to the `class` structure. However, default constructors can cause unintended side effects that produce anomalous or error producing behavior if class developers do not enforce logical requirements.

For example, consider the `Rectangle` class. From a logical standpoint, should it be possible to create a `Rectangle` with a zero value for either `width` or `height`? The answer is no because without a positive value for both dimensions, logically the shape is either a line, a point, nothing, or something that is ill defined, *e.g.* a `Rectangle` with one or more negative values. This logical requirements should be enforced from the moment of `Rectangle` creation and not left up to the user to "fill in later". Part of the class designer's job is to create rules based on logical requirements and use the syntactical features of the language to enforce these requirements so class users cannot work around them. Working around the rules is a significant source of bugs and if a loophole or shortcut in the object's design is left available by a senior class developer, a junior developer or intern will inevitably exploit it which will inevitably lead to a future issue.

To enforce logical state consistency beginning at the very creation of an object, a class developer should ALWAYS provide an explicit constructor. As stated above, the default constructor will not be provided by Java if if any explicit constructor is provided. An explicit constructor can be parameterless or parameterized, *i.e.* a constructor that requires parameters. A class may also have multiple constructors including a no parameter constructor alongside one or more parameterized constructors.

```
class Rectangle {
    int width;
    int height;

    // Explicit parameterless constructor
    Rectangle() {
        width = 0;
        height = 0;
    }
```

```
    // Parameterized constructor
    Rectangle( int w, int h ) {
        width = w;
        height = h;
    }
}
```

However, as suggested above, the above version of the `Rectangle` class is not logically consistent. The parameterless constructor is probably dangerous to include because it means `Rectangle` objects can be created with invalid initial state. A more logically consistent version of the `Rectangle` class would be the following:

```
class Rectangle {
    int width;
    int height;

    // Parameterized constructor
    Rectangle( int w, int h ) {
        width = w;
        height = h;
    }
}
```

This version of the `Rectangle` class eliminates the default constructor and forces class users to call the parameterized constructor. In other words, a class user could no longer create a `Rectangle` instance using the `new Rectangle();` statement. Instead, the class user MUST use the `Rectangle(int w, int h)` constructor to create a `Rectangle` which forces any class user to provide both parameters to the `Rectangle` object when creating every `Rectangle` object. In other words, `new Rectangle(10,20);` would create a `Rectangle` with `width = 10` and `height = 20`. Does this resolve all logical issues? No as it would still be possible to create `Rectangle` instances with zero or negative dimensions (preventing this requires more understand of Java features and more work to implement); however, it does force the class user to provide initial values for required fields which in and of itself will generally reduce and prevent some logical issues related to usage of the `Rectangle` class.

Access Modifiers

Access modifiers are keywords that allow a class designer to intentionally expose or hide class members, *i.e.* methods and instance variables, from class users. There are three access modifiers defined for Java: `public`, `private`, and `protected`. We will only concern ourselves with `public` and `private` access modifiers in this course.

The `public` access modifier is intended to allow a class designer to intentionally expose and grant access to class members, whether instance variables or methods, by a class user. The lazy class developer declares everything with public access to begin with and worries about locking it down later. This is problematic because when a member is exposed, it is not protected and users will bind dependencies to these public members even if the class designer's real intent was to protect these members. This exposes internal class members to outside

manipulation and can result in unexpected and illegal states to arise within an instance. In general, it is bad design to start with everything as `public`, but it is a natural approach because we want convenience and quick development.

The `private` access modifier is intended to allow a class designer to intentionally hide and prevent access to a class member by a class user. The intentional, bug-conscious, safe class developer declares only a very limited set of methods to have public access and otherwise defines all instance variables and all other methods to be private (or protected if relevant) access. Class users should NEVER have direct access to class instance variables such that the internal state of a class should be well-maintained, controlled, and inaccessible to class users. If internal state needs to be exposed to a class user, then the access needs to be well curated and safe. Private access enables support for information hiding which will be discussed later.

Accessors and Mutators

Accessors are commonly referred to as "getters" and mutators are commonly referred to as "setters". This is because they are typically expressed as `get` and `set` class methods. Specifically, these methods are created to explicitly control read and write rights for member instance variables.

An **accessor** is called such because it provides a read access interface for a specific instance variable and a **mutator** (to change) is called such because it provides a write access interface for a specific instance variable. Provided the instance variable is declared as `private`, if only an accessor is provided for an instance variable then the variable is read-only outside the class, if only a mutator is provided for an instance variable then the variable is write-only outside the class, and if both a mutator and an accessor are provided then the instance variable is both read and write outside the class.

This approach gives the class developer the ability to control what is exposed to class users and to only expose prescribed behaviors for a class. It forces class users to use the explicitly designed interface, prevents class users from binding directly to class instance variables that should remain hidden inside the class, and helps ensure that the internal state of the class remains within the design parameters for the class. It also eliminates the burden on class users to have to know how a class works in order to use it properly.

For example, consider a `Circle` class that provides a way for users to get and or set the instance variables and offers the opportunity to enforce validation rules for any mutators provided.

```
class Circle {
    private float radius;

    public float getRadius() {
        return radius;
    }

    public float setRadius( float r ) {
        radius = r;
    }
    ...
}
```

In the above example, the class developer has provided an accessor `getRadius` to allow a class user to read the value of the radius stored inside a `Circle` object. The class developer has also provided a mutator `setRadius` that allows the class user to change the value of the radius. Given sufficient additional rules for the `mutator`, specifically by introducing error throwing, the class designer could prevent a class user from setting the `radius` to 0 or a negative value. If `radius` was declared as `public`, a class user could set the `radius` to zero or a negative value without any supervision or control by the class developer which would produce nonsensical (in this case) or even dangerous state for some classes.

A better implementation might be to not provide a mutator for `radius` at all so that the property cannot be changed after the object is created.

```
class Circle {
    private float radius;

    public Circle(float r) {
        radius = r;
    }

    public float getRadius() {
        return radius;
    }

    ...
}
```

By not providing a mutator, making the `radius` private, and providing a `radius` accessor, it encourages the class user to initialize the class correctly and prevents a class user from modifying the `radius` to an illegal value after the `Circle` object is created.

`this` keyword

The `this` keyword is a convenience reference that allows class developers to clearly specify the context of a symbol that may be used in both local and instance level scopes. According to Java syntax, the same symbol can be declared as an instance variable and as a local variable in the same class and the two variables will be distinct and stored in two different locations in memory, *i.e.* the local variable will be on the stack and the instance variable will be on the heap. This might lead to trying to find different names for variables depending on their scope to avoid collisions and confusion; however, if two variables share the same intent why should we arbitrarily name them something different just to address a scoping difference. For this reason the `this` keyword exists.

`this` acts as a reference to the current instance from within a class. `this` allows a class developer to explicitly refer to an instance level variable from the local scope regardless of whether or not the symbol is declared in local scope. Let's look at the `Circle` example again to clarify.

```
class Circle {
    private float radius;

    public float setRadius( float radius ) {
        radius = radius;  // what is the intent of this line?
                          // what does it actually do?
    }
    ...
}
```

In the above example, the input parameter for the mutator is named `radius` which is the same symbol used for the instance variable `radius`. It is perfectly legal to name these two different variables this way; however, a problem arises with the statement `radius = radius;`. It seems the class developer intends to assign the `radius` instance variable to the value provided by the `radius` input parameter (this is the only context for the example that makes sense); however, this intent is not properly communicated to Java and it will not work the way intended. Instead, Java will interpret the intent of the code is to assign the input parameter to itself and therefore do nothing because that is what the syntax is describing. Java first looks for the symbol in the local scope and if it cannot find it there it then looks in the instance scope, but because there is a `radius` symbol in both scopes, all `radius` labels inside `setRadius` are associated with the local variable. We need some way to differentiate between and specify scope levels at the function level to clearly communicate intent, so how can we provide the intent and avoid renaming the local variable to some arbitrary, less meaningful symbol?

```
class Circle implements Shape {
    private float radius;
    public float setRadius( float radius ) {
        this.radius = radius;
    }
    ...
}
```

The `this` keyword communicates the intent clearly to Java. By using the self reference of `this` in the above example, Java interprets the statement to mean assign the `radius` instance variable the value of the local variable `radius`.

Object Oriented Principles

There are three principles of **Object Oriented Programming (OOP)** : **Encapsulation, Inheritance, Polymorphism**. Since this is a course in data structures, we will naturally and explicitly focus on Encapsulation; however, we will only address Inheritance and Polymorphism around the edges and these topics will be considered in detail in a follow-up Software Engineering course.

Encapsulation

To encapsulate simple means to enclose something. Encapsulation in terms of OOP is to put associated data and methods into a class. For example, most logic needed to store and operate on string data is all encapsulated into the Java String class. This class contains a character array, *i.e.* the string data, and a whole suite of methods for strings that almost fully encapsulate everything associated with strings into a single class.

Whenever you write a class, you are encapsulating related methods and data into that class, so encapsulation is the most fundamental development activity in OOP languages. It allows you to group all the associated operations and data into a single container.

So far, we have assumed that everything generally has public access but this is generally a dangerous assumption. Part of encapsulation is to discriminate if each individual instance variable and method you add to a class should be accessible to everyone or if it should be hidden inside the class. This idea is known as **information hiding** and it is a critical secondary principle that is provided by encapsulation.

Information Hiding

The principle of information hiding is that only the minimal information needed to use the class should be visible outside the class. Users of a class, *i.e.* junior developers or third parties, do not need to know how a class works to use it, they only need to know how to interact with the class and how it behaves in response. This is a form of "black boxing" that allows developers to discriminate between internal and external class data and to protect the internal state of a class from unintended manipulation by a class user. In order to support information hiding, Java provides the `public` and `private` access modifiers which allow for the developer to explicitly specify which variables should be exposed by the class interface. The `protected` access modifier is just a specialized `private` access modifier that assists OOP inheritance.

Inheritance

Inheritance develops relationships between classes based on shared traits. The most obvious example is that you inherit traits from your ancestors and any descendants you have will inherit traits from you and from your ancestral lineage.

For a more abstract example consider that we have circles and rectangles. Circles and rectangles are obviously different, a circle has a radius while a rectangle has a width and height, but a more interesting and maybe more useful organizational question is what do they share in common which we can answer as both are shapes. So, what specifics about shapes are useful for any shape: all shapes have a perimeter derived from its dimensions, all shapes have an area derived from its dimensions, and maybe all shapes can be drawn. This raises the question of how we express all of the above in a set of class structures and whether these relationships can be modeled by the object definitions themselves. There are several possible approaches, but OOP introduces the concept of inheritance to allow OOP languages to model the relationship of shared traits between class definitions and objects.

There are different forms of inheritance in Java and we will not go deeply into them here, but they do involve special keywords. For this form of inheritance we need to use the `interface` keyword to define a `Shape` type and the `implements` keyword for the `Circle` and `Rectangle` classes. We will discuss these keywords in more detail later with respect to Abstract Data Types. The relationships between a generic `Shape` and specific `Circle` and `Rectangle` shapes is illustrated in the following Java code:

```
interface Shape {
    float area();
    float perimeter();
    void draw();
}

class Circle implements Shape {
    float radius;
    float area() {
        return 3.1459 * radius * radius;
    }
    float perimeter() {
        return 3.1459 * 2 * radius;
    }
    void draw() { ... }
}

class Rectangle implements Shape {
    float width;
    float height;
    float area() {
        return width * height;
    )
    float perimeter() {
        return 2 * (width + height);
    }
    void draw() { ... }
}
```

There are many issues with the above code, but it conveys the general idea that a `Shape` defines certain ideas shared by all types of shapes and it is up to a specific shape to implement the specifics of that idea. We will revisit this example below and discuss it in more detail.

Polymorphism

The term polymorphism is the portmanteau of “poly” meaning “many” and “morph” meaning “forms”; therefore, polymorphism means “having many forms”. Ironically, there are many types of polymorphism in programming and non-OOP languages may support some types of polymorphism without even offering a class structure; however, OOP languages specifically offer a form of polymorphism as an extension of inheritance.

One non-OOP form of polymorphism is to have multiple functions with the same label.

```
class Greet {
    static void hello() {
        System.out.println("Hello");
    }
    static void hello( String name ) {
        System.out.println("Hello " + name);
    }
}
```

There are two methods with the same name. How is this possible? This is the common form of polymorphism involving functions. From a practical sense, we use the same symbol to represent two independent operations. Java accepts this because even though these two functions share the same label, the parameter list has a unique permutation of types which is used to discriminate between the same symbols. Java uses the function label and the ordered list of types to find the corresponding function. As suggested above, this is not the only form of polymorphism and you will be introduced to a much more dynamic version once you master inheritance, but this is an important form of polymorphism to discover now as it will free you from trying to invent different symbols for functionally similar operations.

The OOP form of polymorphism is an extension of inheritance. Recall the `Shape`, `Circle`, `Rectangle` example:

```
interface Shape {
    float area();
    float perimeter();
    void draw();
}

class Circle implements Shape {
    float radius;
    float area() {
        return 3.1459 * radius * radius;
    }
    float perimeter() {
        return 3.1459 * 2 * radius;
    }
    void draw() { ... }
}

class Rectangle implements Shape {
    float width;
    float height;
    float area() {
        return width * height;
    )
    float perimeter() {
        return 2 * (width + height);
    }
    void draw() { ... }
}
```

The inheritance here defines that every `Circle` object is both a `Circle` and a `Shape`. Similarly, every `Rectangle` object is both a `Rectangle` and a `Shape`. This means that it is legal for a Shape reference to point to a `Circle`, a `Rectangle`, or any other type that implements the `Shape` interface. Which also means that we can store `Circle`, `Rectangle`, or any other `Shape` references in a data structure defined to hold `Shape` references. If the implications of this are not entirely clear now, it will become more clear as we look at specific data structures.

Abstraction

An **Abstract Data Type (ADT)** is a description of a data type that only prescribes in the most general sense “what it does” but does not specify “how it works”. In other words, an ADT only prescribes the interactions and the behaviors of that type and does not prescribe the implementation which should be generally hidden from class users. An **interaction** is an intentionally exposed interactive element of a system like the button on a device that performs some set of actions in response to user interaction, and a **behavior** describes how the system responds when a specific interaction is evoked. In programming terms, an interaction is a method including a description of the necessary inputs, and a behavior is a high-level generalization of what the method does including a description of any output the system produces in response. These descriptions are extremely abstract and intentionally ignore any specific implementation details about the process.

Consider the following "media player interface":

While a media player user does not necessarily know how the media player works, certain operations have been exposed through buttons and associated icons. For the above interface, we expect that the media player will begin playing whatever the selected media is when the middle button with the right facing triangle is pressed. We recognize this operation as "play" due to its standard interface and as such we have certain expectations of what will happen in response to pressing the button. A description of this interface would be "play" as the interaction and something as simple as "plays the current track" for its associated behavior.

Interfaces become recognizable standards and that systems including humans in society come to depend on. For example, consider the standard interface for driving an automatic car. This interface consists of a steering wheel that rotates clockwise and counter-clockwise, the right pedal, the accelerator, speeds up the car, and the left pedal, the brake, slows down the car. When the car is moving forward, if the steering wheel is turned clockwise the car turns to its right, and if the steering wheel is turned counter-clockwise the car turns to its left. Even if the same controls are used but their layout or behaviors were changed by car designers, it would have dangerous effects on the driver and people around the car where drivers would have to relearn the car interface for each car they drove. For example, suppose that some cars turned left when the steering wheel was turned clockwise or some cars had the accelerator on the left. It is because the car interface is standard that drivers can easily and safely drive unfamiliar cars.

The term **interface** is generally used to describe the interactive elements of a system specifically where two systems interact and exchange information. The term interface, much like the term state, can be used to describe a single element or can be used to describe an entire system. We can focus on a function's interface such as the **function signature** which includes the return type, label, and list of parameter types, we can focus on a class

interface which includes the public methods and public instance variables, we can focus on an application's interface such as the inputs, outputs, buttons, windows, and other controls used to interact with an application, or we could use the term to focus on complex systems like a computer to describe all the interactive elements such as keyboard, monitor, mouse, etc and the connections joining other subsystems like USB, SATA, etc.

As illustrated by the abstract `Shape` definition, Java even uses `interface` as a keyword for an abstract class:

```
interface Shape {
    float area();          // computes (returns) the geometric area of the shape
    float perimeter();     // computes (returns) the geometric perimeter of the shape
    void draw(Window w);   // draws the shape to a graphical Window device
}
```

An abstract class is a class that only defines the interface for a type and does not provide any implementations. In other words, it defines only how to interact with the type and how the type will respond to specific interactions. It does not define the implementation meaning it does not define how the type will actually function to achieve the stated interface. In the `Shape` `interface` above there are only function declarations meaning that none of the functions provide implementation details. As a result, a Java `inteface` cannot be created directly meaning that attempting to create an instance of an interface would be illegal, *e.g.* using this statement `Shape s = new Shape();` would lead to a compilation error. Instead a class would have to "implement" the `interface` as illustrated by the following `Circle` class and an instance of the Circle could be created:

```
class Circle implements Shape {
    float radius;

    // computes (returns) the geometric area of the shape
    float area() {
        return 3.1459 * radius * radius;
    }

    // computes (returns) the geometric perimeter of the shape
    float perimeter() {
        return 3.1459 * 2 * radius;
    }

    // draws the shape to a graphical Window device
    void draw(Window w) { ... }
}
```

Note that Java uses the `implements` keyword to establish the relationship between the `interface` definition and a class. This is a contract where a class that `implements` the `interface` must define implementations for all methods in the associated `interface`. In the `Circle` class, the class must provide implementations for `area()`, `perimeter()`, and `draw(Window)` because the class is declared as `implements Shape`. As mentioned earlier, this is a form of inheritance and helps support polymorphism at the class level.

A natural question for this Java interface model is what are the benefits of developing these associations? Consider the following example program:

```java
class ShapeAppEx {

    void main( String[] args ) {
        // create an array of abstract shapes
        Shape[] shapes = new Shapes[2];

        // create specific shapes and store references to them in the shape array
        shapes[0] = new Circle( 1.0 );
        shapes[1] = new Rectangle( 2.0, 4.0 );

        // pass the abstract array to a function that operates on them
        draw(shapes);
    }

    // iterates over all shapes and draw each one
    void draw( Shape[] shapes ) {
        Window w = System.getWindow();  // Note this is just a conceptual example

        for( int i = 0; i < shapes.length; i++ ) {
            Shape shape = shapes[i];
            shape.draw( w );
        }
    }
}
```

Recall that conceptually everything must be an object in an object oriented language which is why all code in Java must be contained within classes. When a programmer defines a class, they are defining an object. Java reflects this defining every class to inherit from an `Object` base class. The `Object` class defines the default `toString()` method that is inherited by all objects which generally allows any custom type to be printed using `System.out.print(...)` or `System.out.println(...)`. In other words, when you print any reference type, you are calling the `toString()` method associated with that type. If there is no `toString()` method provided in the class definition, the default `Object.toString()` method will be called.

`Circle c = new Circle(1.0);` `System.out.println( c );`	→	`Circle c = new Circle(1.0);` `System.out.println( c.toString() );`

The class designer does not have to provide the `toString()` method for their class because it is already provided in the `Object` class; however, the default behavior of the `Object.toString()` method may not be useful for specialized classes. The Java 8 documentation describes the `Object.toString()` implementation as:

> The `toString` method for class `Object` returns a string consisting of the name of the class of which the object is an instance, the at-sign character '`@`', and the unsigned hexadecimal representation of the hash code of the object. In other words, this method returns a string equal to the value of:
>
> ```java
> getClass().getName() + '@' + Integer.toHexString(hashCode())
> ```

> Object.toString() Java 8 documentation

In other words, the default `Object` class `toString()` method returns the name of the class and some additional information which is not generally useful to the average programmer. Instead, a programmer might want instead the `toString()` method for their own class to return a string containing detailed state information about an object so that the programmer can peak inside the class and trace state data.

For example, in a simple `Point` class, we could override the default `toString()` method:

```
class Point {
    private float x;
    private float y;

    Point(float x, float y) {
        this.x = x;
        this.y = y;
    }

    // overrides the inherited Object class method toString and returns
    // information involving the instance variables of this circle
    public String toString() {
        return "Point(" + "x=" + x  + ",y=" + y + ")";
    }
}
```

From the above, we can create a simple program where we create a point, and then call the system print operation which in turn implicitly calls the `Point` class `toString()` method:

```
class PointToStringEx {
    public static void main( String[] args ) {
        Point pt = new Point( 1.0f, 2.0f );
        System.out.println( "pt=" + pt );
    }
}
```

If we run the above, it should print the following to the console:

```
pt=Point(x=1.0,y=2.0)
```

In the `System.out.println( "pt=" + pt );` instruction, we are asking Java to build a string consisting of the `"pt="` prefix concatenated with whatever the output of `pt`'s class definition for its `toString()` method. If we had not overridden the `Point` class `toString()` method by providing one in the `Point` class itself, it would have used the `Object` class `toString()` method instead which would have simply printed according to the Java 8 definition discussed above. However, we did provide our own `Point` class `toString()` method which is defined to build a `String` defined as `"Point(" + "x=" + x  + ",y=" + y + ")"` which contains detailed state information about this instance. The `toString()` method can be customized to the programmer's and program's needs.

The inheritance used to define the relationship between the `Object` class and all other classes is slightly different from the inheritance defined by the `interface` and `implements` model illustrated above. The details of these differences will be left for another class to explore, but it is important to understand that OOP allows programmers to define relationships between classes and to inherit entire methods or just the interfaces from another class.

List ADT

The first step in the development of any class is to determine the set of interactions and their resultant behaviors for that class and to determine what these interactions are we have to first characterize the class itself. In other words, if we want to define a class that encapsulates the entire idea of a list, we have to first address the question of "what is a list".

A list is a generic tool we use to structure information in our daily lives. When we think of a list, we might think of a shopping list, a todo list, or a contact list. Evidently all of these things are lists because we call them that, but to classify (build a class) that captures the critical essence of all lists, we need to identify what all lists share in common. Let's work with a very concrete example of a shopping list and use it to try to identify characteristics of a list.

Assuming we have a piece of paper and a pen handy, we have an empty list ready for use. Great, but it is mostly useless as an empty list so the first thing we usually do is add a new element to the list, so we need an interaction that allows us to add an element to the list. What is the associated behavior that add produces? Do we just write anywhere on the list? For practical reasons when the paper gets full, we might, but we generally do not start out that way and we only do that because of physical limitations, e.g. too little remaining space on the page, need to reorder the list but we wrote in pen, etc. No, add for a list is generally thought of as append, or in other words, we typically add a new element to the end of the list. So this suggests our first and possibly most important method shared by all lists:

```
add( element )  # append element to the end of the list
```

Note that this is intentionally a pseudo code to avoid the specifics of Java right now. It allows us to avoid specific implementation considerations like types when sketching this out as a concept. We want the idea of the list to be as generic and language independent as possible to start out with and we will worry about how it works when we actually try to write it in a specific language (implement) like Java.

So when writing down a shopping list of items, I generally add items in the order I think of them without too much consideration for things like shopping order. For example, I might think of eggs, milk, bread, cereal so I will probably add things to my list in that order:

```
add( eggs )
add( milk )
add( bread )
add( cereal )
```

Which generates the following list:

1. eggs

2. milk
3. bread
4. cereal

Note that lists implicitly have an order that emerges simply by writing down each element one after another.

At its minimum we expect a list to have certain positional characteristics, specifically, a list guarantees both the absolute positioning and relative positioning of elements with respect to the list and other elements in the list. In short, a list is a special kind of set where a linear order is imposed based upon position and when a linear order is imposed, positioning naturally emerges. **Absolute position** means that an element has a unique position within the set based upon a numerical index, *i.e.* first, second, last. **Relative position** means that an element has a position relative to other elements in the set, *i.e.* the first element precedes all other elements, the second element follows the first element but is before the remaining elements, the last element follows all other elements. If a set is structured by absolute position, then relative position naturally emerges, and if a set is structured by relative position, then absolute position naturally emerges. This brings us to another method for a list, accessing an element by positon:

```
get( position )    # get element by absolute position in the list
```

However, just because we have the ability to use absolute position in all lists, only lists that are specifically optimized around absolute position will support efficient interactions using absolute position. In other words, arrays are better tools if your list needs to be accessed by absolute position because arrays support constant time access of any element in an array based list, but a list structured around relative positioning will not be as efficient as an array if absolute positioning access is used. So while get is a generic list interaction, whether it is wise to use it frequently will depend on whether the list implementation efficiently supports the interaction.

In addition to adding and retrieving elements from the list, we also typically need a way to remove elements from the list. In reality, we may need a variety of ways to remove elements from a list, *e.g.* remove by absolute position, remove by relative position, remove by some other identifier, however, let's keep this simple to start out with and suggest that when we shop, we look at the top-most element of the list, go find it, and scratch it off (remove from) our list. Then, we can simply define remove as:

```
remove( )    # remove the first element in the list if it exists
```

We also may need other basic information like how many elements are in our list. This tracks with basic questions we might ask about a list like "how long is my list", but it is also very practical from an implementation standpoint because to support iteration using our defined get we probably need to know the length of the list. This leads to another interaction:

```
size( )     # answers question of how many elements are in list
```

Finally, we might want to ask questions about whether or not something is in our shopping list. Like remove, we might have a variety of different forms of this question, but in the simplest terms, when looking at items in a store, we might simply need to answer the question of "is this item on my list". Let's generalize all possible search questions into a single method to start with:

```
contains( element ) # answers question of whether element is in list
```

This gives us an overall generic interface for a list:

```
add( element )  # append element to the end of the list
get( position )    # get element by absolute position in the list
remove( )    # remove the first element in the list if it exists
size( )     # answers question of how many elements are in list
contains( element ) # answers question of whether element is in list
```

Yes we may be omitting additional interactions that fully define a list, but as an example, we are really focusing on the essential operations necessary to support the abstract idea of list. By listing too much we also may start to drift into other ideas that are not necessarily list related which may confuse and crowd this simple idea with too many features. There is nothing preventing us from adding new interactions later as we discover them and classify them as list operations, but it is often a good design decision to keep things simple and to let complexity emerge later.

For example, we might be tempted to add a variety of remove interactions to support all the different ways removal needs to be supported. But we can actually still support something like remove by absolute position if we just call remove i times. This would remove everything preceding the element to be removed, so we could use add for each element removed preceding the i-th element to ensure these elements are put back into the list. It may be inefficient and it may result in the list being reordered, but this approach still fulfills the general behavior of any remove(position) interaction we might define.

Interface

We use the term **interface** to generally describe interactions and behaviors. An interface can be thought of as a location where two or more entities interact. Generally we want to provide prescribed interfaces that facilitate positive, specific, and protected actions between entities. We may use the term about a single method, as in the method interface is just another way to describe the **method signature**, *i.e.* the method label, return value, and list of input parameters. We may use the term about a class, as in the class interface is a way to describe all the methods defined for the class, or the public class interface, as in all the methods exposed to outside entities with the public access modifier that allow interaction with the class.

In Java there is also the interface keyword which meets all the above definitions but also has a very narrowly tailored definition. Recall the earlier definition of the Shape class:

```
interface Shape {
    float area();
    float perimeter();
    void draw();
}
```

This may seem odd, but it is a legal definition in Java and is in fact how we define ADTs in Java. Remember that an abstract data type defines "what it does" and not "how it works". In other words, an ADT only defines the interface, *i.e.* the method declarations for that data structure, and does not prescribe the implementation, *i.e.* the actual method definitions. The implementations are left up to any classes that fulfill (implement), the interface which brings us to the rest of the example:

```
class Circle implements Shape {
    float radius;
    float area() {
        return 3.1459 * radius * radius;
    }
    float perimeter() {
        return 3.1459 * 2 * radius;
    }
    void draw() { ... }
}

class Rectangle implements Shape {
    float length;
    float width;
    float area() {
        return length * width;
    }
    float perimeter() {
        return 2 * (length + width);
    }
    void draw() { ... }
}
```

The `Circle` and `Rectangle` classes use the implements keyword to declare to Java that these classes will fulfill the interface established by `Shape`. In order to be a `Shape`, a class must implement the Shape interface. Which probably raises the question of why would we do this and the answer is both inheritance and polymorphism.

According to the example, any `Circle` object is also a `Shape` and likewise any `Rectangle` object is also `Shape`. This provides a number of benefits:

- We design a list that holds generic `Shape` types instead of two different types of lists, *e.g.* `RectangleList` and `CircleList`.
- We can pass any `Circle` instance and any `Rectangle` instance as a `Shape` parameter without needing to know if it is a `Circle` or a `Rectangle`.
- If we find we are using the wrong implementation later, we can change out a type declaration without needing to rewrite everything because all entities use the same interface.

This last point is not a great example with shapes, but it is a good example when dealing with lists. Consider the following example:

```
interface List {
    void append( Object o );        // append an element to the end of the list
    Object get( int i );            // get a reference to list element by absolute position
    Object remove( int i );         // remove an element by absolute position
    int length();                   // get the length of the list
    boolean contains( Object o );   // determine if a specific element exists in the list
}

class ArrayList implements List { ... }
class LinkedList implements List { ... }
```

If both `ArrayList` and `LinkedList` classes fulfill the `List` interface without exposing additional methods or any of their instance variables, then the only way to interact with these entities is by using the `List` interface. This means you can change out one data structure for the other at a later time without worrying about breaking anything.

For example, suppose we started development and chose an `ArrayList` for our implementation:

```
class ListApp {
    public static void main( String args[] ) {
        ArrayList list = new ArrayList();
        list.append( new Element(1) );
        list.append( new Element(2) );
        list.append( new Element(3) );
        list.remove( 1 );
        for( int i = 0; i < list.length(); i++ ) {
            Object o = list.get( i );
            ...
        }
        ...
    }
}
```

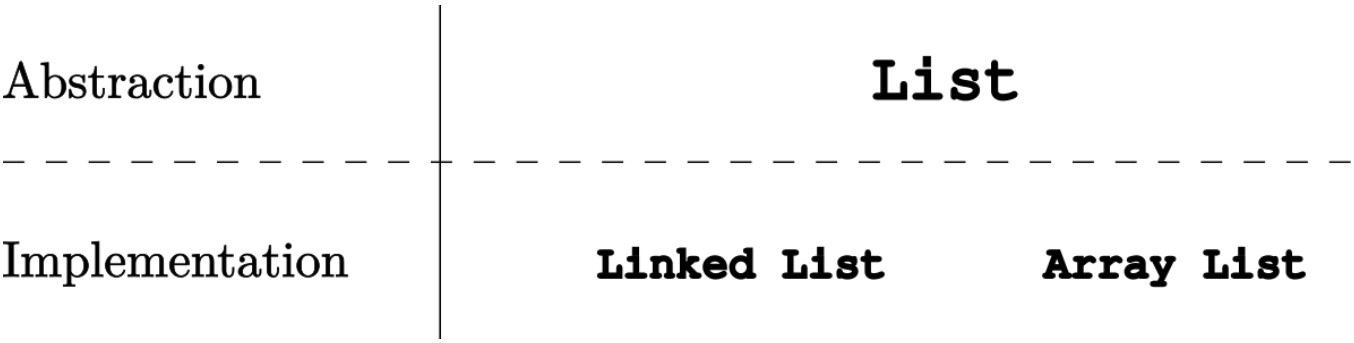
Suppose we discover months after release that the `ArrayList` was a poor choice and we really needed to use a `LinkedList`, if we did not share the same interface and did not adhere to it, we might need to make hundreds or thousands of changes to our code, which could result in months of debugging and potential disaster if we are not thorough in our reassessment of the system. However, if we used the abstract data type as a template and we stuck to it rigorously, all of those hours of redeveloping and debugging and the massive risk of bulk changes might be reduced to a small, single change:

```
class ListApp {
    public static void main( String args[] ) {
        LinkedList list = new LinkedList();
        list.append( new Element(1) );
        list.append( new Element(2) );
        list.append( new Element(3) );
        list.remove( 1 );
```

```
        for( int i = 0; i < list.length(); i++ ) {
            Object o = list.get( i );
            ...
        }
        ...
    }
}
```

Layering

The purpose of the ADT is to intentionally create layers that hide the implementation away from users. Class users engage with the abstraction layer and if it is well defined they don't need to know anything about how the class is supported to use it.



Users of the class only see the public interface and these are the only interactions available to them. As far as a class user is concerned a linked list is indistinguishable from an array list in terms of how to use it because they have the same external interface. The details are intentionally hidden away within the classes so that class users cannot manipulate the internal state of an instance without validation.

We limit what is accessible to class users by using access modifiers to limit what users can interact with inside the class. If the instance variables are hidden from class users by `private` declarations and well reasoned and controlled accessors and mutators are the only means to access the internal state of the object, then class users can only interact with methods intentionally made `public` which prevents class users from binding to implementation specific variables, like the array backing an array list, and ensures that class users must use the prescribed interface.

It is not necessary to know how something works in order to use it if it has a well designed interface. We can hide the implementation from users to protect the system and users from poor usage, like how the hood of a car keeps one from putting their hands inside a running engine. A system that hides the implementation from the user is called a "black box".

Black Boxing

A **black box** is a system that we don't need to see inside a box in order to use it. By making an opaque "black" skin around the box and providing only specific ports and buttons on its surface, a designer protects internal components and provides the supported interfaces to the user. This limits how the box can be used and ensures the internal state inside the box remains consistent with any design assumptions.

Any provided external interfaces, i.e. buttons like a power button, screen, keyboard, etc. or ports like USB or lightning connector, prescribe the legal and supported actions and features of a device. These interfaces represent the expected, supported, and sometimes legal ways a system is to be used.

When you attempt to open up a physical device, you often encounter various tamper indicators, stickers or fasteners, designed to be destroyed if the device has been opened after sale. These tamper indicators allow a manufacturer to determine if the box has been opened by a user. A manufacturer typically protects its liabilities through legal contracts, e.g. a warranty or end user agreement, that specifies the box never be opened and these tamper devices help ensure the manufacturer can detect tampering and contract violations.

The skin of a black box plays an important role. It prevents users from interacting with the object in ill-advised or even dangerous ways. If your phone did not have a case around it and all the internal wiring was exposed, you could easily short out the battery or damage other components just by touching the phone in the wrong way. For this reason we also want to provide a skin for classes. We can skin a class, i.e. hide information, if we generally assume every instance variable and method is private to begin with and only set those methods that we explicitly approve for use by a class user to be `public`. It requires much more developmental work, but for large systems, it is imperative to limit and control access to internal class components so that the class performs as advertised and cannot be corrupted by user interference with internal state.

Generic Classes (Templated Types)

Throughout the material of this course there is one question that constantly lurks in the shadows, how do we implement a data type that can support all types. So far, in each of our examples the class designer has specified a type for data stored in a class, but is there a way to design a class such that the class user defines the type stored in the class. Java is built on inheritance and we have the `Object` type as the base class at the top of the object hierarchy, so we might try to specify data generically as an `Object`.

This is object reference polymorphism. If every class inherits from the `Object` base class, then every type is both its specified type and `Object` type. To create an object of specialized type, we have to declare it of that type and create it using the associated constructor; however, once the instance is created, we can cast its reference to any type that is in its inheritance hierarchy.

Our `List` interface would therefore define any methods that require the type of the data to be specified as base class `Object` types. For example:

```
interface List {
    void append( Object o );
    Object get( int i );
    Object remove( int i );
    int length();
    boolean contains( Object o );
}
```

On the implementation side, any classes that implement the List interface would fulfill the List interface using the base class `Object` type.

```
class ArrayList implements List {
    Object[] data;
    int size;

    ArrayList( int size ) {
        data = new Object[size];
        this.size = size;
    }

    void add( Object o ) { ... }
    Object get( int i ) { ... }
    Object remove( int i ) { ... }
    int length( ) { ... }
    boolean contains( Object o ) { ... }
    ...
}
```

This would allow programs that interact with any `List` derived class to pass base class Object references to the list; however, it would require the class user to know what type each Object stored in the list is in order to cast it to its correct type when ready to use it.

```
ArrayList list = new ArrayList(10);
// The Integer references can be implicitly downcast to Object types since the
// Integer class inherits from the Object class
list.append( new Integer(1) );
list.append( new Integer(2) );
// get returns a reference to an Object type, so to use it as an Integer type
// the returned reference needs to be explicitly upcast to Integer.  It can
// get complicated even with only a single type in the List to cast appropriately
// every time we interact with the list.
int x = ((Integer) list.get(1)).intValue();
```

Instead of using the `Object` base class we could instead ask the class user to provide a type when they declare a `List` using generics similar to how we defined generic functions. Instead, we will define a generic class:

```
interface List<T> {
    void append( T o );
    T get( int i );
    T remove( int i );
    int length( );
    boolean contains( T o );
}
```

In the above interface, we are no longer explicitly specifying the type of the data stored in the list until the list is declared. The implementation side uses similar syntax:

```
class ArrayList< T > implements List<T> {
    T[] data;
    int size;

    ArrayList( int size ) {
        data = new T[size];
        this.size = size;
    }

    T get( int i ) {
        if( i < 0 || i >= size) {
            return null;
        }
        for(int i = 0; i < size; i++) {
            return data[i];
        }
    }

    void append( T o ) { ... }
    T remove( int i ) { ... }
    int length( ) { ... }
    boolean contains( T o ) { ... }

    ...
}
```

Review

1. What is the role of a class?
2. When is a constructor called?
3. What does the `new` keyword do?
4. When is the `new` keyword used?
5. What always follows the `new` keyword?

6. What are the characteristics of a constructor?
7. What is the default constructor?
8. What happens to the default constructor when an explicit constructor is defined?
9. Why should you provide an explicit constructor?
10. Why does it make your class less safe when you rely on the default constructor?
11. What is a parameterless constructor?
12. What is a parameterized constructor?
13. What is an access modifier?
14. Why should you generally make instance variables private?
15. What is a mutator? Is a mutator read-only or write-only?
16. What is an accessor? Is an accessor read-only or write-only?
17. How can you combine mutators and accessors to control read and write access to instance variables?
18. What does it mean for an instance variable to have the `public` keyword in its declaration? What if it has the `private` keyword instead?
19. What is an object?
20. What is an instance?
21. What is the relationship between instance variables and an instance?
22. What does the `this` keyword represent?
23. How can this be used to change the scoping level of two variables with the same name but with different scopes, *i.e.* from local to instance?
24. What does the word abstract mean in the context of computer science?
25. Explain the concept of *encapsulation* in the context of OOP.
26. Explain the concept of *information hiding* in the context of programming.
27. Explain the concept of *inheritance* in the context of OOP.
28. Explain the concept of *polymorphism* in the context of OOP.
29. Why do we create abstraction layers?
30. What is meant by the term *interface*? What is meant by the terms *interactions* and *behaviors* with respect to an interface?
31. What does the Java keyword `interface` mean? What is its relationship to a class?
32. How does the Java interface relate to the `implements` keyword?
33. What does Abstract Data Type mean?
34. How is an abstract type typically expressed in Java?
35. What is the minimal List interface?
36. What are the characteristics of a list?
37. What do we mean by *absolute position*?

38. What do we mean by *relative position*?

39. What do we mean by “the list is an abstraction”?